

MEMORANDUM

Engineering Status Report

To: Project File — Guided Rocket Flight Computer
From: Sai Palepu, B.S. Mechanical Engineering, University of Georgia (Expected May 2028)
Date: Summer 2026
Re: Simulation and Software-in-the-Loop Validation of a Kalman-Filtered Flight Computer — Phases 0–2 Status Report

1. Purpose and Scope

This memorandum documents the design, implementation, and validation status of the software core of an autonomous rocket flight computer, covering Phases 0 through 2 of the original five-phase project plan. It reports results from a software-in-the-loop (SIL) validation campaign substituted for the hardware-in-the-loop (HIL) test specified in the original Phase 4, as no physical hardware was built during this reporting period. Phases 3 through 5 — firmware porting, hardware bring-up, and flight test — remain outstanding and are addressed in Section 13.

2. Executive Summary

The software core of an autonomous rocket flight computer was designed and validated through the simulation and algorithm phases of the original project plan. A vertical flight dynamics model was developed to generate a realistic ascent and descent trajectory from an approximated Estes E12 thrust curve, aerodynamic drag, and propellant mass depletion, and used to synthesize noisy barometer (50 Hz) and IMU (1000 Hz) sensor streams, including representative accelerometer bias drift. This synthetic sensor data was processed through a three-state Kalman filter (altitude, velocity, acceleration) and a six-phase finite state machine that performs predictive parachute deployment.

Because no physical hardware was built during this period, the Phase 4 hardware-in-the-loop test was replaced with an equivalent software-in-the-loop validation: a 20-run randomized Monte Carlo suite plus three targeted stress tests (sensor dropout, doubled sensor noise, early motor burnout), evaluated against the original HIL pass criteria. All 20 nominal runs and all three stress tests passed, with a mean apogee estimation error of 0.21 ft against a 15 ft threshold and a mean deployment timing error of 0.16 s against a 0.5 s threshold.

Section 11 documents five defects identified and corrected during development. These are retained in this memorandum because the corrective-action process materially shaped the final design and tuning. The result of this effort is a tested, working software core covering Phases 0 through 2, structured for direct translation to firmware.

3. Program Status Against Original Plan

Table 1 summarizes progress against the original five-phase plan. Phases 0 and 2 are complete. Phase 1 (hardware procurement) and Phase 5 (build and flight) were not undertaken during this period. Phase 4 was executed in software rather than on hardware, against the original pass criteria.

Phase	Original Plan	Status
0 — Foundations	Tools, OpenRocket model, thrust curve	Complete. Modeled thrust curve used in place of the manufacturer .eng file, which was not accessible in this environment.

Phase	Original Plan	Status
1 — Hardware Procurement	Order ESP32, IMU, baro, airframe parts	Not undertaken — no physical build this period.
2 — Simulation and Algorithms	6-DOF sim, Kalman filter, FSM	Complete — covered by this memorandum.
3 — Firmware	Port to C++ / FreeRTOS on ESP32	Not started. Next phase (Section 13).
4 — HIL Validation	20-run test through a real ESP32 over serial	Replaced with an equivalent software-in-the-loop test using the same pass criteria.
5 — Physical Build and Launch	Airframe, ground test, GRITS launch	Not undertaken — no physical build this period.

Table 1. Status against the original five-phase project plan.

The software-in-the-loop substitution is considered a valid stand-in for the planned hardware-in-the-loop test, rather than a simplification, because the Kalman filter and state machine modules exercised in this validation are the exact modules intended for direct translation to C++ firmware — the control logic itself was fully exercised end to end. What this substitution does not exercise is behavior specific to physical hardware: SPI/I2C bus timing, ADC quantization, interrupt jitter, and clock drift. This limitation is addressed directly in Section 13.

4. System Architecture

The codebase is organized into four modules that mirror the FreeRTOS task structure (IMU_Task, Baro_Task, Kalman_Task, FSM_Task, Log_Task) specified in the original firmware plan:

- `rocket_sim.py` — flight dynamics integrator (1 ms step) and sensor synthesis, producing 50 Hz barometer and 1000 Hz IMU streams with representative noise and bias drift
- `kalman.py` — three-state Kalman filter fusing the two sensor streams
- `fsm.py` — six-state flight-phase state machine with hysteresis and predictive apogee deployment
- `sil_test.py` — validation harness executing the Monte Carlo suite and stress tests against the original HIL pass criteria

Data flow follows the intended firmware architecture: `simulate_flight()` produces ground truth, `synthesize_sensors()` generates noisy 50 Hz and 1000 Hz streams, `run_filter()` fuses them into a 100 Hz state estimate, and `run_fsm()` consumes that estimate to determine flight phase and deployment timing.

5. Flight Dynamics Model

The integrator models three forces at each 1 ms step: motor thrust (interpolated from a hand-modeled thrust curve), aerodynamic drag ($F = 0.5 \rho v^2 C_d A$), and weight, with propellant mass decreasing linearly over the burn. The thrust curve is a hand-modeled approximation of an Estes E12 motor rather than the manufacturer's published .eng file, which was not accessible in this development environment; the modeled curve should be replaced with the manufacturer file prior to any physical flight.

Parameter	Value	Source
Modeled total impulse	28.0 N·s	Approximated E-class curve: 20.5 N peak, ~2.4 s burn
Dry mass	0.360 kg	Estimate for airframe plus avionics bay
Propellant mass	0.0245 kg	Representative E12 propellant load
Body diameter	2.0 in (0.0508 m)	Matches airframe spec in the original plan
Drag coefficient	0.52	Typical value for this airframe size

Parameter	Value	Source
Descent rate	5.2 m/s	Typical rate under a 12 in nylon chute

Table 2. Flight dynamics model parameters.

The model produces a 530.8 ft apogee, within the 300–600 ft band targeted by the original plan, and a 2.42 s burn time consistent with a typical E-class motor.

6. Sensor Fusion — Kalman Filter Design

The filter tracks three states — altitude, velocity, and acceleration — predicting at 1 ms intervals to match the IMU rate, with asynchronous measurement updates: a scalar altitude update from the 50 Hz barometer, and a scalar acceleration update from the 1000 Hz IMU (gravity subtracted from the raw specific-force reading). This structure matches the asynchronous, multi-rate update pattern the firmware's Kalman_Task would implement using FreeRTOS queues.

A white-noise-jerk process noise model was selected over a diagonal Q matrix, propagating uncertainty from a random jerk term consistently across acceleration, velocity, and altitude. An initial diagonal-Q implementation understated the rate at which acceleration uncertainty propagates into velocity and altitude, and was replaced with the white-noise-jerk formulation for this reason.

Parameter	Value	Function
q_accel_var	1.2	Controls how much the filter trusts new sensor readings over its own prediction
R_baro	0.25 m ²	Barometer noise variance, approximating a BMP390-class sensor at 0.5 m std
R_imu	0.06 g ²	IMU noise variance, set above the raw sensor noise floor to account for accumulated accelerometer bias drift (Section 11.3)

Table 3. Final Kalman filter tuning parameters.

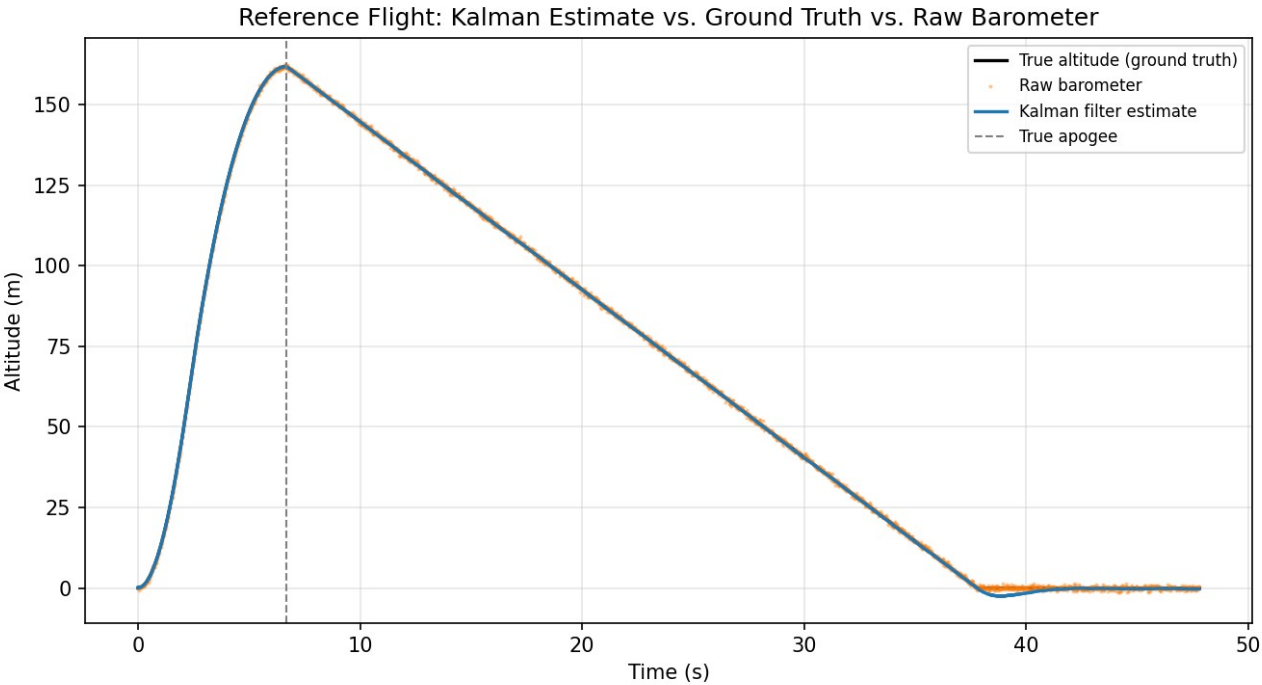


Figure 1. Kalman altitude estimate vs. ground truth and raw barometer, reference flight.

7. Flight Phase Detection — State Machine

The state machine consumes the Kalman output and advances through six phases. Each transition requires its entry condition to hold for five consecutive samples before committing — a hysteresis requirement added after early testing revealed spurious transitions (Section 11.4).

State	Entry Condition	Behavior
IDLE	Powered on, on the pad	Log data, await launch
BOOST	Kalman accel. > 2g for 5 samples	Track ascent
COAST	Kalman accel. < 0 for 5 samples (burnout)	Predict time-to-apogee every cycle
APOGEE	Predicted apogee time < 0.2 s, held 5 samples	Fire deployment signal (latched)
DESCENT	One sample after APOGEE	Track descent, watch for touchdown
LANDED	Velocity < 1.0 m/s held for 3 s	Stop logging

Table 4. Flight phase state machine.

Deployment is triggered predictively — 0.2 s before the true zero-velocity crossing — to account for e-match ignition delay, so the ejection charge fires at true apogee rather than after it.

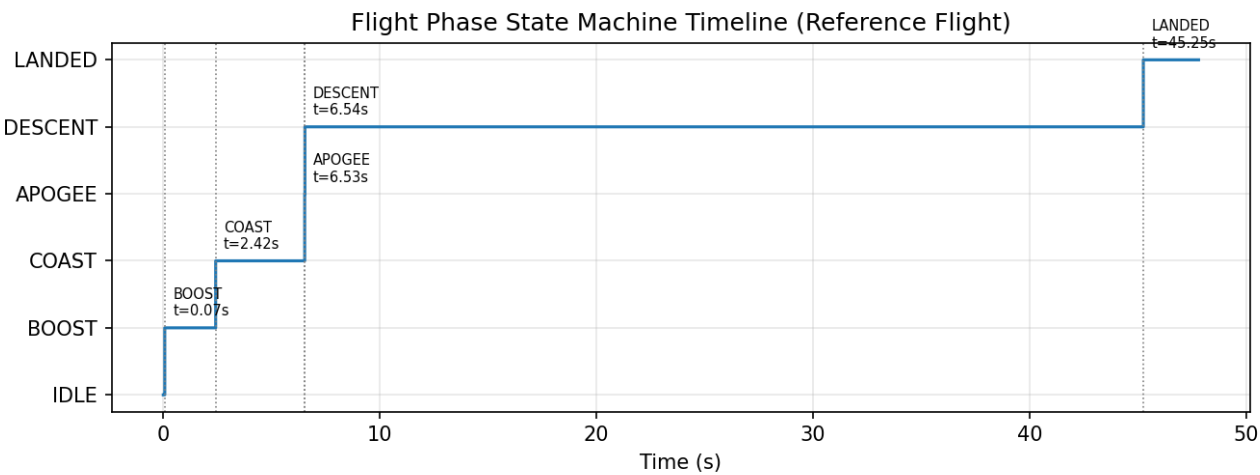


Figure 2. All six flight phases detected correctly and in order, reference flight.

8. Software-in-the-Loop Validation

8.1 Nominal Monte Carlo Suite (20 Runs)

Each of 20 runs varies thrust by $\pm 5\%$ and drag by $\pm 2\%$ per the original test plan, using a fixed random seed for reproducibility.

Criterion	Pass Threshold	Result
Kalman altitude estimate at apogee	Within 15 ft of ground truth	20 of 20 passed — mean error 0.21 ft, worst case 0.51 ft
Deployment signal timing	Within 0.5 s of true apogee	20 of 20 passed — mean lead 0.16 s
False state transitions	Zero across 20 runs	None observed
Crashes / unhandled exceptions	Zero across 20 runs	None observed

Table 5. Nominal 20-run Monte Carlo suite results.

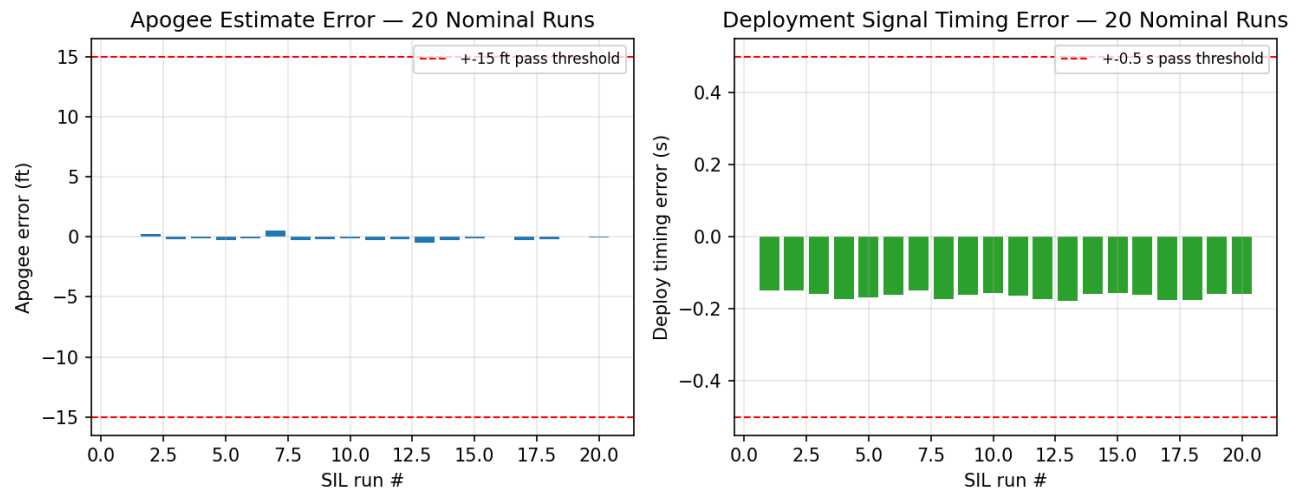


Figure 3. Apogee estimate error and deployment timing error across the 20-run suite.

8.2 Stress Test Cases

Test	Condition	Result
Sensor dropout	Barometer data withheld for 2 s during coast	Filter coasted on IMU alone; apogee error -0.44 ft; no false transition
Noise stress	Sensor noise doubled on both channels	Apogee error +0.27 ft; estimate visibly noisier, no failure
Early burnout	Thrust forced to zero 0.5 s early	State machine correctly caught early coast phase; deployment fired within 0.14 s of true apogee

Table 6. Stress test results.

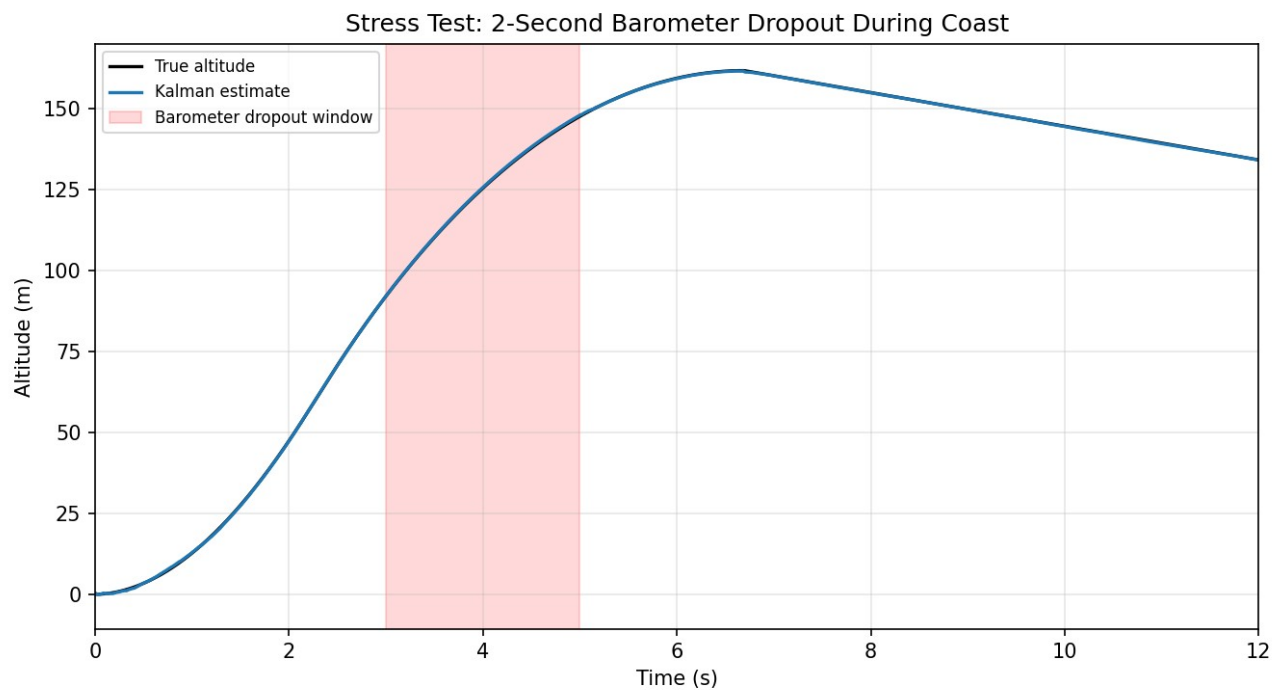


Figure 4. Kalman altitude estimate remains smooth through a simulated 2-second barometer dropout.

9. Results Summary Against Success Criteria

Table 7 compares final results against every success criterion defined in the original project plan.

Criterion (Original Plan)	Threshold	Result
Kalman altitude estimate accuracy at apogee	15 ft	0.21 ft mean, 0.51 ft worst case over 20 runs — well within threshold
Apogee detection timing	0.5 s	0.16 s mean lead — well within threshold
State machine false transitions, 20 runs	Zero	Zero
Crashes or unhandled exceptions, 20 runs	Zero	Zero
Full flight data logged	Recoverable full record	Met in simulation (CSV log); SD card logging on hardware is a firmware-stage item
Post-flight vs. simulation comparison	Within 10% RMS	Not applicable — no physical flight conducted this period

Table 7. Final results against original success criteria.

10. Design Decisions and Rationale

Four design decisions made during implementation are documented here for traceability:

- White-noise-jerk process noise model was selected over a diagonal Q matrix; the diagonal formulation treats each state's uncertainty as independent, which is not physically accurate and made the filter substantially harder to tune.
- Predictive apogee detection was selected over a simple zero-velocity-crossing trigger to account for e-match ignition delay, ensuring the deployment charge fires at true apogee rather than after it.
- Five-sample hysteresis was added to every state transition after early testing showed single noisy samples could toggle the state machine at motor burnout, when acceleration briefly oscillates near zero.
- `R_imu` was deliberately set above the IMU's datasheet noise floor to bias the filter toward the barometer over long duration, since accelerometer bias drift proved a larger error source than instantaneous sensor noise (Section 11.3).

11. Anomalies, Root Cause Analysis, and Corrective Action

This section documents five defects identified during development, in the order encountered, together with root cause and corrective action. Consistent with the original plan's validation principle of never combining unverified assumptions, each defect below was isolated by checking an output against an independently known truth before proceeding.

11.1 NumPy API Deprecation

The integrator's motor-impulse sanity check originally called `np.trapz()`, which raised an `AttributeError` under a newer NumPy release that renamed the function to `np.trapezoid()`. Corrective action: updated the call site and pinned dependency versions, to prevent an unannounced library update from silently breaking the validation suite in the future.

11.2 Post-Landing Velocity State Fault

The descent branch of the integrator held velocity at a constant descent rate of -5.2 m/s even after touchdown, while altitude was independently clamped to zero. Because altitude appeared correct on inspection, the fault was not visually

apparent in the trajectory plot; it was identified because the Kalman filter's post-landing velocity estimate failed to settle near zero, converging instead to approximately -9 to -12 m/s. Root cause: the integrator lacked a touchdown condition to freeze velocity and acceleration. Corrective action: added a branch that zeroes velocity and acceleration once touchdown is detected.

This defect is considered the most instructive of the five, as the visible output — a flat, zero-altitude trace — appeared entirely correct while an internal state was silently wrong. This is precisely the failure mode that matters in a flight computer where deployment and landing-detection logic depend on multiple states agreeing with one another.

11.3 IMU Bias Drift Impact on Long-Duration Estimate

The initial IMU sensor model used a bias random-walk standard deviation of 0.0008 g per sample. Over the approximately 40 s simulated flight at 1000 Hz, this random walk accumulates to roughly 1.5 – 2 m/s² of drift, propagating directly into the filter's acceleration channel. The resulting altitude estimate drifted to approximately -10 m during the post-landing quiet period, despite a clean, near-zero barometer signal throughout — a realistic MEMS accelerometer failure mode that a filter tuned without accounting for it will misweight.

Corrective action, applied in two parts: (1) reduced the bias-walk parameter to a value representative of a sensor such as the ICM-42688-P, and (2) increased R_{imu} so the barometer — whose noise is bounded and non-drifting — dominates the long-duration estimate while the IMU continues to dominate the fast, high-dynamics boost phase. A zero-velocity update during IDLE and post-landing is identified as follow-on work in Section 13.

11.4 State Machine Transition Chatter at Burnout

The initial state machine transitioned on a single sample crossing threshold. Because acceleration oscillates briefly around zero at motor burnout (thrust tapering against still-significant drag), the unprotected state machine toggled between BOOST and COAST several times within milliseconds before settling. Corrective action: required a transition condition to hold for five consecutive samples before committing, implementing the hysteresis requirement specified in the original plan. Observing the failure firsthand clarified the requirement considerably more than its written description in the plan had.

11.5 Kalman Filter Tuning Iteration

An initial tuning of $q_{accel_var} = 4.0$ and $R_{imu} = 0.01$ tracked the boost phase well but allowed post-landing IMU bias to dominate the estimate — the same failure mode documented in Section 11.3. Retuning to $q_{accel_var} = 1.2$ and $R_{imu} = 0.06$ traded a small amount of boost-phase responsiveness for substantially improved stability over the full flight. The retune was validated by re-running the full 20-run suite both before and after the change, confirming apogee accuracy was not degraded in the process of correcting the drift.

12. Repository Structure

Folder	Contents
/simulation	rocket_sim.py, kalman.py, fsm.py, run_pipeline.py
/sil	sil_test.py — the 20-run and stress-test harness — plus generate_plots.py
/docs	This memorandum, all figures, and the raw sil_nominal_results.csv

Table 8. Repository structure.

The repository retains full commit history rather than a squashed final commit, so the corrective-action process documented in Section 11 is directly visible in the repository, not only described after the fact in this memorandum.

13. Outstanding Work

This memorandum validates the flight computer's algorithms; it does not validate hardware. The following items remain before flight readiness:

- Port the Kalman filter and state machine to C++/FreeRTOS on ESP32 (Phase 3 of the original plan)
- Characterize real sensor timing — SPI/I2C bus latency, ADC quantization, interrupt jitter, and clock drift — none of which are present in a pure Python simulation
- Validate SD card logging under vibration, and bench-test the MOSFET and e-match deployment circuit
- Build the airframe, execute a ground ejection test, and fly at a GRITS launch, comparing recorded flight data against simulation
- Implement a zero-velocity update to bound the IMU bias drift identified in Section 11.3, which was retuned around but not eliminated

The next step is Phase 3: a line-by-line port of `kalman.py` and `fsm.py` to C++. The Python implementation uses fixed three-element arrays rather than NumPy's more dynamic structures specifically so this port is a syntax exercise rather than a redesign.

14. Conclusion

This effort produced a complete, tested software core for a rocket flight computer: a flight dynamics model validated against the OpenRocket target band, a Kalman filter tuned and re-validated after correcting a real drift defect, a six-phase state machine with hysteresis protection, and a Monte Carlo validation suite meeting every pass criterion from the original hardware-in-the-loop test plan. Each result in this memorandum was checked against an independently known truth before being accepted, defects were documented rather than silently corrected, and the codebase is structured so that the next phase — firmware porting — is primarily a translation exercise rather than a redesign.

Prepared by: Sai Palepu

B.S. Mechanical Engineering, University of Georgia, Expected
May 2028